

LAPORAN PRAKTIKUM

SISTEM OPERASI

PERTEMUAN 11

(Memori Xinu)



Oleh:

(Izzaty Zahara Br Barus)

(2311104052)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

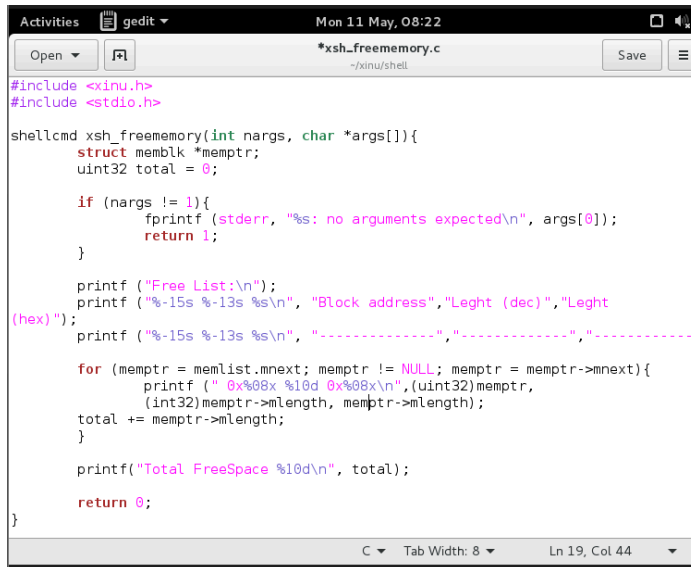
DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

Jurnal

1. [80 poin] Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.



```
#include <xinu.h>
#include <stdio.h>

shellcmd xsh_freememory(int nargs, char *args[]){
    struct memblk *memptr;
    uint32 total = 0;

    if (nargs != 1){
        fprintf(stderr, "%s: no arguments expected\n", args[0]);
        return 1;
    }

    printf("Free List:\n");
    printf("%-15s %-13s %s\n", "Block address", "Leght (dec)", "Leght
(hex)");
    printf("%-15s %-13s %s\n", "-----", "-----", "-----");

    for (memptr = memlist.mnext; memptr != NULL; memptr = memptr->mnext){
        printf(" 0x%08x %10d 0x%08x\n", (uint32)memptr,
            (int32)memptr->mlength, memptr->mlength);
        total += memptr->mlength;
    }

    printf("Total FreeSpace %10d\n", total);

    return 0;
}
```



```
{
    { "argcho",      xsh_argcho},
    { "arp",         xsh_arp},
    { "cat",         xsh_cat},
    { "clear",       xsh_clear},
    { "date",        xsh_date},
    { "devdump",     xsh_devdump},
    { "echo",        xsh_echo},
    { "freememory",  xsh_freememory},
    { "help",        xsh_help},
    { "ls",          xsh_ls},
    { "kill",        xsh_kill},
    { "memdump",     xsh_memdump},
    { "memstat",     xsh_memstat},
    { "ns",          xsh_ns},
    { "netinfo",     xsh_netinfo},
    { "ping",        xsh_ping},
    { "ps",          xsh_ps},
    { "sleep",       xsh_sleep},
    { "tee",         xsh_tee},
    { "udp",         xsh_udpdump},
    { "udpecho",     xsh_udpecho},
    { "udpeserver",  xsh_udpeserver},
    { "uptime",      xsh_uptime},
    { "mycmd",       xsh_mycmd},
    { "namecmd",     FALSE, xsh_namecmd},
    { "",            xsh_help}
}
```



```
*shprototypes.h
~xinu/include

extern shellcmd xsh_help      (int32, char *[]);

/* in file xsh_kill.c */
extern shellcmd xsh_kill     (int32, char *[]);

/* in file xsh_ls.c */
extern shellcmd xsh_ls       (int32, char *[]);

/* in file xsh_led.c */
extern shellcmd xsh_led       (int32, char *[]);

/* in file xsh_memdump.c */
extern shellcmd xsh_memdump   (int32, char *[]);

/* in file xsh_memstat.c */
extern shellcmd xsh_memstat   (int32, char *[]);

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory (int nargs, char *[]);

/* in file xsh_netinfo.c */
extern shellcmd xsh_netinfo   (int32, char *[]);

/* in file xsh_ns.c */
extern shellcmd xsh_ns        (int32, char *[]);

/* in file xsh_nvram.c */
extern shellcmd xsh_nvram     (int32, char *[]);
```

```
2311104052_development-system [Running] - Oracle VirtualBox : 1
File Machine View Devices Help
Activities Terminal Mon 11 May, 08:25
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Tabs Help
xinu@xinu-develop-end: ~/... xinu@xinu-develop-end: ~/... xinu@xinu-develop-end: ~/...

-----Izzaty zahara Br Barus-----
XINU
-----2311104052-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address Leght (dec) Leght (hex)
-----
0x0018e110 -975128 0xffff1ee8
0x00100000 5079040 0x004d8000
0x005e8000 49152 0x0000c000
Total FreeSpace 4153064
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

2. [4 poin per subsoal] Jawablah pertanyaan berikut:

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu memisahkan data segment dan BSS segment karena keduanya memiliki karakteristik yang berbeda:

- Data segment menyimpan variabel global yang sudah diinisialisasi (memiliki nilai awal), sehingga nilainya harus disimpan secara eksplisit di dalam file executable/binary.
- BSS segment menyimpan variabel global yang belum diinisialisasi, sehingga tidak perlu menyimpan nilainya di dalam binary cukup dicatat ukurannya saja, dan saat program dimuat ke memori, segmen ini diisi nol secara otomatis.

Pemisahan ini bertujuan untuk menghemat ukuran file binary/executable: variabel yang belum diinisialisasi tidak perlu memakan ruang di disk karena hanya perlu menyimpan informasi ukurannya. Ini membuat loading program menjadi lebih efisien.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi mempengaruhi ukuran free space?

Selama eksekusi Xinu:

- Setiap kali proses baru dibuat, sistem akan mengalokasikan sebagian dari free space untuk stack proses tersebut (via `getstk()`), sehingga ukuran free space berkurang.
- Setiap kali proses terminasi, stack proses tersebut dilepas kembali ke free space (via `freestk()`), sehingga ukuran free space bertambah kembali.
- Untuk heap, proses yang memanggil `getmem()` akan mengambil sebagian free space, dan `freemem()` mengembalikannya. Namun heap tidak otomatis dibebaskan saat proses terminasi, sehingga jika tidak di-free secara eksplisit, free space dapat terus berkurang seiring berjalannya program (memory leak).

Kesimpulannya: ukuran free space bersifat dinamis dan terus berubah sesuai aktivitas alokasi dan dealokasi proses maupun heap.

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
- **Dealokasi tidak otomatis** heap harus dibebaskan secara manual menggunakan `freemem()`. Jika proses terminasi tanpa membebaskan heap, maka terjadi memory leak.
 - **Tidak terstruktur** heap bisa dialokasikan dan dealokasi kapan saja secara bebas, tidak seperti stack yang terstruktur LIFO sehingga lebih mudah diprediksi.
 - **Fragmentasi** akibat alokasi dan dealokasi yang acak, free space bisa terpecah menjadi blok-blok kecil yang tidak kontigu sehingga sulit mengalokasikan blok besar.
 - **Bisa melebihi free space** tidak ada jaminan heap yang diminta selalu tersedia, jika free space habis maka `getmem()` akan gagal.
- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
- **Dealokasi tidak otomatis** heap harus dibebaskan secara manual menggunakan `freemem()`. Jika proses terminasi tanpa membebaskan heap, maka terjadi memory leak.
 - **Tidak terstruktur** heap bisa dialokasikan dan dealokasi kapan saja secara bebas, tidak seperti stack yang terstruktur LIFO sehingga lebih mudah diprediksi.
 - **Fragmentasi** akibat alokasi dan dealokasi yang acak, free space bisa terpecah menjadi blok-blok kecil yang tidak kontigu sehingga sulit mengalokasikan blok besar.
 - **Bisa melebihi free space** tidak ada jaminan heap yang diminta selalu tersedia, jika free space habis maka `getmem()` akan gagal.
- e. Apa tantangan utama dalam penggunaan heap di Xinu?
- Tantangan utama dalam penggunaan heap di Xinu antara lain:
1. **Memory Leak** Heap tidak dibebaskan secara otomatis saat proses terminasi. Proses harus secara eksplisit memanggil `freemem()`. Jika lupa, memori bocor dan tidak bisa digunakan kembali.
 2. **Fragmentasi Memori** Akibat alokasi dan dealokasi heap yang tidak beraturan, free space dapat terpecah menjadi blok-blok kecil yang tidak kontigu. Meskipun total free space mencukupi, permintaan alokasi blok besar bisa gagal karena tidak ada blok yang cukup besar secara kontigu.
 3. **Permintaan melebihi kapasitas** Tidak ada jaminan bahwa heap yang diminta akan selalu tersedia. Jika free space habis akibat banyak alokasi, permintaan `getmem()` bisa gagal.
 4. **Kompleksitas manajemen** Programmer harus secara cermat mengelola siklus hidup memori heap (alokasi dan dealokasi), yang menambah kompleksitas pengembangan program di Xinu.